

RESUMO DA EXECUCAO DO PROJETO DE QA

Sistema avaliado: Desafio Virtual - Corra por Voce

Finalidade: apresentar o que foi desenvolvido e executado para atender as solicitacoes da atividade de Garantia da Qualidade de Software (QA), incluindo os comandos usados na demonstracao pratica.

Tecnologia	Python / VS Code
Regras de negocio documentadas	7
Casos de teste automatizados	10
Relatorios	HTML - normal e com bug proposital
Demonstracao de defeito	QA-008 - calculo de progresso

1. Atendimento ao enunciado

A atividade solicitou um pequeno projeto de QA em Python, executado no Visual Studio Code, com sistema/contexto definido, pelo menos 4 funcionalidades ou regras, no minimo 8 casos de teste, registro automatico dos resultados, PASS/FAIL, severidade, metricas, decisao de QA, relatorio automatico e demonstracao de um bug proposital.

Foi utilizado o projeto real **Desafio Virtual - Corra por Voce**. A automacao ficou isolada na pasta **qa/** do repositorio.

2. Estrutura implementada

Arquivo	Responsabilidade
qa/main.py	Executa testes, calcula metricas, decide o status de QA e gera relatorio.
qa/regras.py	Regras de negocio avaliadas.
qa/casos_teste.py	10 casos de teste com esperado, obtido, PASS/FAIL e severidade.
qa/gerador_relatorio.py	Geracao automatica dos relatorios HTML.
qa/README.md	Documentacao da automacao e da demonstracao.

3. Regras e casos de teste

Foram avaliadas regras de validacao de e-mail, duplicidade, acesso do participante, quilometragem, progresso, evidencia da atividade e ranking.

ID	Teste	Esperado	Severidade
QA-001	Aceitar e-mail valido	True	Alta
QA-002	Rejeitar e-mail invalido	False	Alta
QA-003	Detectar e-mail duplicado	True	Critica
QA-004	Permitir acesso cadastrado	True	Critica
QA-005	Bloquear e-mail nao cadastrado	False	Critica
QA-006	Aceitar km valida	True	Alta
QA-007	Bloquear km negativa	False	Critica
QA-008	Calcular percentual de progresso	50.0	Critica
QA-009	Validar atividade com evidencia	True	Alta
QA-010	Ordenar ranking por km	Atleta B	Media

4. Como executar os testes no VS Code

Abra o terminal do VS Code na raiz do projeto. Se ainda nao estiver dentro da pasta **qa**, execute:

```
cd qa
```

Teste da versao normal:

```
python main.py
```

Resultado esperado: 10 testes executados, 10 PASS, 0 FAIL, taxa de aprovacao de 100%, nenhuma falha critica e decisao **LIBERADO**.

Teste com bug proposital:

```
python main.py --bug
```

Esse comando ativa intencionalmente o defeito preparado para a atividade. O QA-008 deve falhar porque o progresso esperado e 50.0, mas o modo com bug retorna 25.0.

5. Resultado da execucao normal

Indicador	Resultado
Testes executados	10
PASS	10
FAIL	0
Taxa de aprovacao	100,0%
Falhas criticas	0

DECISAO FINAL: LIBERADO

6. Resultado esperado ao testar o bug

Indicador	Normal	Com bug
Testes executados	10	10
PASS	10	9
FAIL	0	1
Taxa de aprovacao	100,0%	90,0%
Falhas criticas	0	1
QA-008	PASS	FAIL
Decisao	LIBERADO	NAO LIBERAR

No terminal, a falha relevante deve aparecer de forma semelhante a:

```
QA-008 | FAIL | Calcular percentual de progresso  
Esperado: 50.0  
Obtido: 25.0  
Severidade: CRITICA
```

```
Total de testes: 10  
Aprovados: 9  
Falharam: 1  
Taxa de aprovacao: 90.0%  
Falhas criticas: 1  
DECISAO FINAL DE QA: NAO LIBERAR
```

7. Relatorios e conclusao

A execucao gera automaticamente **relatorio_qa_normal.html** e **relatorio_qa_bug.html**. Os relatorios registram os casos, esperado, obtido, PASS/FAIL, severidade, metricas e decisao final.

O projeto demonstra o fluxo solicitado na atividade: **Requisito -> Teste -> Execucao -> Resultado -> Dados -> Identificacao de Defeitos -> Decisao de QA -> Relatorio**. A comparacao entre as duas

execucoes evidencia que a automacao identifica uma regressao critica e impede a liberacao da versao defeituosa.